Project_FSB_Estatement_Transaction (1)

October 20, 2026

1 e-Statement

e-Statement (electronic statement) yaitu laporan atau dokumen yang disajikan dalam bentuk digital. Dalam konteks perbankan, e-Statement biasanya berupa laporan transaksi rekening secara elektronik yang berisi informasi seperti tanggal transaksi, jenis transaksi, nominal, saldo, dan riwayat aktivitas rekening.

E-Statement merupakan laporan transaksi rekening dalam bentuk digital yang membutuhkan data akurat, terintegrasi, dan mudah diakses. Karena data e-Statement berasal dari berbagai sumber, diperlukan Data Mart e-Statement untuk mengintegrasikan, mengolah, dan menyediakan data yang siap digunakan untuk reporting.

Project ini bertujuan meningkatkan efisiensi, konsistensi, akurasi, dan performa dalam proses penyediaan e-Statement.

2 A. Proses Ekstraksi Data

Data bersumber dari Google spreadsheet dengan link berikut. Data tersebut merupakan data transaksi lengkap pada FSB Bank seperti setor atau tarik tunai, transfer, pembelian barang atau jasa dan seterusnya. Lakukan proses ekstraksi data untuk setiap data master ataupun data transaksinya

Task A.1 - Import library yang dibutuhkan

[3]:
```
# Import library
import pandas as pd
```

[4]:
```
# Setting agar kolom bisa tampil semua
# Memuat file Excel
excel_path = 'work/FSB Bank Transaction (1).xlsx'
xls = pd.ExcelFile(excel_path)

# Melakukan perulangan untuk setiap sheet
for sheet in xls.sheet_names:
    df = pd.read_excel(excel_path, sheet_name=sheet)
    print(f"=== Sheet: {sheet} (Total Baris: {len(df)}) ===")

    # Menampilkan 5 baris pertama
    display(df.head(5))
```

```
    print("\n" + "="*80 + "\n")
```

```
=== Sheet: transaction (Total Baris: 1640) ===

      transaction_id  transaction_reference     account_number  \
0  TRX21325483579612  100001020252132548357  90000260801008087
1  TRX21335043336597  100001020252133504333  90000260801002198
2  TRX21343446066904  100001020252134344606  90000260801006643
3  TRX21357196529986  100001020252135719652  90000260801008644
4  TRX21439859637972  100001020252143985963  90000260802006117

  debit_or_credit transaction_code  transaction_date  transaction_time  \
0               C           TRX001           2025213            112726
1               C           TRX001           2025213            142301
2               C           TRX001           2025213            111501
3               C           TRX001           2025213            105331
4               C           TRX001           2025214            121626

    amount                           transaction_remark  channel
0  5200000    SETORTUNAI VIA CASH DEPOSIT MACHINE 00047  ATM_CDM
1  4000000                    SETORTUNAI VIA AGENT 00070    AGENT
2  7600000  SETORTUNAI VIA CASH RECYCLING MACHINE 00037  ATM_CRM
3  3100000                    SETORTUNAI VIA AGENT 00183    AGENT
4  8400000    SETORTUNAI VIA CASH DEPOSIT MACHINE 00070  ATM_CDM

================================================================================

=== Sheet: master_account (Total Baris: 123) ===

      account_number cif_number account_open_date account_currency_type  \
0  90000260806004444  CIF009116        2025-08-06                   IDR
1  90000260819009735  CIF000329        2025-08-19                   IDR
2  90000260801002198  CIF092706        2025-08-01                   IDR
3  90000260823007873  CIF037812        2025-08-23                   IDR
4  90000260821002646  CIF037510        2025-08-21                   IDR

  account_status product_code             product_name
0         ACTIVE    TABPLTMRB  TABUNGAN UTAMA PLATINUM
1         ACTIVE    TABEMPWAD  TABUNGAN UTAMA KARYAWAN
2         ACTIVE    TABGLDMRB      TABUNGAN UTAMA GOLD
3         ACTIVE    TABGLDMRB      TABUNGAN UTAMA GOLD
4         ACTIVE    TABGLDMRB      TABUNGAN UTAMA GOLD

================================================================================

=== Sheet: master_customer (Total Baris: 123) ===

  cif_number   customer_name gender cif_open_date  branch_code  \
```

```
0  CIF009116  Samiah Pratama      P    2026-08-06        50001
1  CIF000329   Ajeng Hasanah      P    2026-08-19        50002
2  CIF092706   Kania Puspita      P    2026-08-01        20001
3  CIF037812   Fitria Namaga      P    2026-08-23        20002
4  CIF037510   Raina Wahyuni      P    2026-08-21        50003

  customer_birth_place customer_birth_date  \
0       Bandar Lampung          1967-06-11
1                Medan          1996-08-29
2              Bandung          1966-01-23
3             Sukabumi          1988-09-26
4         Tebingtinggi          1968-11-18

                        customer_address   customer_city       customer_phone  \
0   Jl. Cihampelas No. 0, Bandar Lampung  Bandar Lampung   +62 (282) 987-0841
1       Jl. Ir. H. Djuanda No. 85, Medan           Medan   +62 (029) 521-5123
2            Gang Jamika No. 42, Bandung         Bandung           0872801911
3      Gg. Moch. Ramdan No. 29, Sukabumi        Sukabumi  +62 (0628) 579-5264
4  Jalan M.H Thamrin No. 4, Tebingtinggi    Tebingtinggi  +62 (0686) 791 9545

         occupation married_status               customer_email
0            Dokter        Widowed  samiah_pratama@outlook.co.id
1          Pedagang       Divorced     ajeng_hasanah@yahoo.co.id
2  Ibu Rumah Tangga         Single       kania_puspita@gmail.com
3          Pedagang        Married       fitria_namaga@yahoo.com
4         Mahasiswa       Divorced       raina_wahyuni@gmail.com

================================================================================

=== Sheet: master_transaction_code (Total Baris: 6) ===

  transaction_code transaction_desc
0           TRX001     Cash Deposit
1           TRX002         Transfer
2           TRX003  Cash Withdrawal
3           TRX004         Purchase
4           TRX091      Monthly Fee

================================================================================

=== Sheet: master_bank_code (Total Baris: 38) ===

   bank_code bank_swift_code                    bank_name
0          8        BMRIIDJA                 Bank Mandiri
1          9        BNINIDJA  Bank Negara Indonesia (BNI)
2          2        BRINIDJA  Bank Rakyat Indonesia (BRI)
3         14        BBBAIDJA      Bank Central Asia (BCA)
4         11        BDINIDJA       Bank Danamon Indonesia
```

```
================================================================================

=== Sheet: master_branch (Total Baris: 74) ===

   branch_code        branch_name branch_location
0        50001  KC Bandar Lampung  Bandar Lampung
1        50002           KC Medan           Medan
2        20001         KC Bandung         Bandung
3        20002        KC Sukabumi        Sukabumi
4        50003    KC Tebingtinggi    Tebingtinggi

================================================================================
```

Task A.2 - Proses Ekstraksi Data

- Buat fungsi untuk proses ekstraksi data di gsheet, beri nama fungsi *extract_data_from_gsheet* dengan 3 parameter yakni sheet_id, sheet_name dan dtype (buat dtype secara default None)
- Inisialisasi sheet_id data
- Ekstrak data pada masing - masing sheet lalu inisialisasikan ke masing - masing variabel
- Pastikan dtype untuk data master_bank_code terisi mapping {'bank_code' = pd.StringDtype()} dikhawatirkan pandas akan membaca kode bank sebagai integer (periksa datanya di google sheet)
- Inspeksi informasi umum pada data dan identifikasi primary key, foreign key dan juga informasi tambahan lainnya

[5]:
```python
# Fungsi untuk ekstraksi data (file Excel lokal)
def extract_data_from_gsheet(sheet_id, sheet_name, dtype=None):
    excel_file = 'work/FSB Bank Transaction (1).xlsx'
    return pd.read_excel(excel_file, sheet_name=sheet_name, dtype=dtype)

# Inisialisasi sheet_id
sheet_id = '1He4b-15DsAJG_dwYgBz5nDVNW_zQfXZ3VSYoAsr4Wvo'

# Ekstrak data pada masing-masing sheet ke dalam variabel terpisah
transaction = extract_data_from_gsheet(sheet_id, 'transaction')
master_account = extract_data_from_gsheet(sheet_id, 'master_account')
master_customer = extract_data_from_gsheet(sheet_id, 'master_customer')
master_transaction_code = extract_data_from_gsheet(sheet_id,␣
 ↪'master_transaction_code')

# Pastikan dtype untuk master_bank_code terisi mapping {'bank_code': pd.
 ↪StringDtype()}
master_bank_code = extract_data_from_gsheet(
    sheet_id,
    'master_bank_code',
    dtype={'bank_code': pd.StringDtype()}
```

```
)

master_branch = extract_data_from_gsheet(sheet_id, 'master_branch')

print("Semua data berhasil diekstrak dan diinisialisasi ke variabel␣
 ↪masing-masing!")
```

```
Semua data berhasil diekstrak dan diinisialisasi ke variabel masing-masing!
```

Task A.3 - Inspeksi informasi umum pada data

- Data Transaction

[6]:
```
# Tampilkan sampel data transaction
print("=== Sampel Data Transaction ===")
display(transaction.head(2))

print("\n" + "="*50 + "\n")

# Inspeksi informasi umum pada data transaction
print("=== Informasi Umum / Info Data Transaction ===")
transaction.info()
```

```
=== Sampel Data Transaction ===
```

	transaction_id	transaction_reference	account_number	debit_or_credit	transaction_code	transaction_date	transaction_time	amount	transaction_remark	channel
0	TRX21325483579612	100001020252132548357	90000260801008087	C	TRX001	2025213	112726	5200000	SETORTUNAI VIA CASH DEPOSIT MACHINE 00047	ATM_CDM
1	TRX21335043336597	10000102025213350433	90000260801002198	C	TRX001	2025213	142301	4000000	SETORTUNAI VIA AGENT 00070	AGENT

```
==================================================

=== Informasi Umum / Info Data Transaction ===
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1640 entries, 0 to 1639
Data columns (total 10 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   transaction_id        1640 non-null   object
 1   transaction_reference 1640 non-null   object
 2   account_number        1640 non-null   int64
```

```
 3   debit_or_credit        1640 non-null   object
 4   transaction_code       1640 non-null   object
 5   transaction_date       1640 non-null   int64
 6   transaction_time       1640 non-null   int64
 7   amount                 1640 non-null   int64
 8   transaction_remark     1640 non-null   object
 9   channel                1640 non-null   object
dtypes: int64(4), object(6)
memory usage: 128.3+ KB
```

Primary Key : transaction_id Foreign Key : account_number, transaction_code Dimensi Data : 1640 baris dan 10 kolom

Pada data transaction perlu dilakukan transformasi data pada kolom transaction_date & transaction_time hal ini dilakukan karena type data tidak standar, perlu diubah menjadi tipe data datetime agar valid untuk analisis waktu, pengurutan, maupun proses filtering ke database PostgreSQL.

- Data Master Account

[7]:
```
# Tampilkan sampel data master_account
print("=== Sampel Data Master Account ===")
display(master_account.head(2))

print("\n" + "="*50 + "\n")

# Inspeksi informasi umum pada data transaction
print("=== Informasi Umum / Info Data Master ccount ===")
transaction.info()
```

```
=== Sampel Data Master Account ===
```

	account_number	cif_number	account_open_date	account_currency_type	account_status	product_code	product_name
0	90000260806004444	CIF009116	2025-08-06	IDR	ACTIVE	TABPLTMRB	TABUNGAN UTAMA PLATINUM
1	90000260819009735	CIF000329	2025-08-19	IDR	ACTIVE	TABEMPWAD	TABUNGAN UTAMA KARYAWAN

```
==================================================

=== Informasi Umum / Info Data Master ccount ===
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1640 entries, 0 to 1639
Data columns (total 10 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   transaction_id         1640 non-null   object
```

```
 1   transaction_reference  1640 non-null   object
 2   account_number         1640 non-null   int64
 3   debit_or_credit        1640 non-null   object
 4   transaction_code       1640 non-null   object
 5   transaction_date       1640 non-null   int64
 6   transaction_time       1640 non-null   int64
 7   amount                 1640 non-null   int64
 8   transaction_remark     1640 non-null   object
 9   channel                1640 non-null   object
dtypes: int64(4), object(6)
memory usage: 128.3+ KB
```

[8]:
```
# Inspeksi informasi umum pada data master_account
# Mengubah tipe data account_number dari int64 menjadi string/object
master_account['account_number'] = master_account['account_number'].astype(str)

# Cek kembali info DataFrame
master_account.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 123 entries, 0 to 122
Data columns (total 7 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   account_number         123 non-null    object
 1   cif_number             123 non-null    object
 2   account_open_date      123 non-null    datetime64[ns]
 3   account_currency_type  123 non-null    object
 4   account_status         123 non-null    object
 5   product_code           123 non-null    object
 6   product_name           123 non-null    object
dtypes: datetime64[ns](1), object(6)
memory usage: 6.9+ KB
```

Primary Key : account_number Foreign Key : cif_number dan product_code Dimensi Data : 123 baris dan 7 kolom

Pada data master_account perlu dilakukan transformasi data pada kolom account_number hal ini dilakukan karena perlu dirubah dari type integer ke type string(object)

- Data Master Customer

[9]:
```
# Tampilkan sampel data master_customer
print("=== Sampel Data Master Customer ===")
display(master_customer.head(2))

print("\n" + "="*50 + "\n")
```

```
=== Sampel Data Master Customer ===
```

```
  cif_number   customer_name gender cif_open_date  branch_code  \
0  CIF009116  Samiah Pratama      P    2026-08-06        50001
1  CIF000329   Ajeng Hasanah      P    2026-08-19        50002

  customer_birth_place customer_birth_date  \
0       Bandar Lampung          1967-06-11
1                Medan          1996-08-29

                      customer_address   customer_city      customer_phone  \
0  Jl. Cihampelas No. 0, Bandar Lampung  Bandar Lampung  +62 (282) 987-0841
1      Jl. Ir. H. Djuanda No. 85, Medan           Medan  +62 (029) 521-5123

  occupation married_status                customer_email
0     Dokter        Widowed  samiah_pratama@outlook.co.id
1   Pedagang       Divorced     ajeng_hasanah@yahoo.co.id


==================================================
```

[10]:
```
# Inspeksi informasi umum pada data master_customer
# Mengubah tipe data branch_code dari int64 menjadi string/object
master_customer['branch_code'] = master_customer['branch_code'].astype(str)

# Cek kembali info DataFrame
master_customer.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 123 entries, 0 to 122
Data columns (total 13 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   cif_number            123 non-null    object
 1   customer_name         123 non-null    object
 2   gender                123 non-null    object
 3   cif_open_date         123 non-null    datetime64[ns]
 4   branch_code           123 non-null    object
 5   customer_birth_place  123 non-null    object
 6   customer_birth_date   123 non-null    datetime64[ns]
 7   customer_address      123 non-null    object
 8   customer_city         123 non-null    object
 9   customer_phone        123 non-null    object
 10  occupation            123 non-null    object
 11  married_status        123 non-null    object
 12  customer_email        123 non-null    object
dtypes: datetime64[ns](2), object(11)
memory usage: 12.6+ KB
```

Primary Key : cif_number Foreign Key : branch_code Dimensi Data : 123 baris dan 13 kolom

Pada data master_customer perlu dilakukan transformasi data pada kolom branch_code hal ini dilakukan karena perlu dirubah dari type int64 ke type string (obejct)

- Data Master Transaction Code

[11]:
```
# Tampilkan sampel data master_transaction_code
print("=== Sampel Master Transaction Code ===")
display(master_transaction_code.head(5))

print("\n" + "="*50 + "\n")

# Cek kembali info DataFrame
master_transaction_code.info()
```

```
=== Sampel Master Transaction Code ===
```

	transaction_code	transaction_desc
0	TRX001	Cash Deposit
1	TRX002	Transfer
2	TRX003	Cash Withdrawal
3	TRX004	Purchase
4	TRX091	Monthly Fee

```
==================================================

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 2 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   transaction_code  6 non-null      object
 1   transaction_desc  6 non-null      object
dtypes: object(2)
memory usage: 228.0+ bytes
```

Primary Key : transaction_code Foreign Key : … Dimensi Data : 6 baris dan 2 kolom

- Data Master Bank Code

[12]:
```
# Tampilkan sampel data master_bank_code
print("=== Sampel Master Transaction Code ===")
display(master_bank_code.head(5))

print("\n" + "="*50 + "\n")

# Cek kembali info DataFrame
master_bank_code.info()
```

```
=== Sampel Master Transaction Code ===
```

```
  bank_code bank_swift_code                    bank_name
0       008        BMRIIDJA                 Bank Mandiri
1       009        BNINIDJA  Bank Negara Indonesia (BNI)
2       002        BRINIDJA  Bank Rakyat Indonesia (BRI)
3       014        BBBAIDJA      Bank Central Asia (BCA)
4       011        BDINIDJA       Bank Danamon Indonesia


==================================================

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 38 entries, 0 to 37
Data columns (total 3 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   bank_code        38 non-null     string
 1   bank_swift_code  38 non-null     object
 2   bank_name        38 non-null     object
dtypes: object(2), string(1)
memory usage: 1.0+ KB
```

Primary Key : bank_code Foreign Key : bank_swift_code Dimensi Data : 38 baris dan 3 kolom

- Data Master Branch

[13]:
```
# Tampilkan sampel data master_branch
print("=== Sampel Master Branch ===")
display(master_branch.head(5))

print("\n" + "="*50 + "\n")

# Cek kembali info DataFrame
master_branch.info()
```

```
=== Sampel Master Branch ===

   branch_code        branch_name branch_location
0        50001  KC Bandar Lampung  Bandar Lampung
1        50002          KC Medan           Medan
2        20001         KC Bandung         Bandung
3        20002        KC Sukabumi        Sukabumi
4        50003    KC Tebingtinggi    Tebingtinggi


==================================================

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 74 entries, 0 to 73
Data columns (total 3 columns):
 #   Column           Non-Null Count  Dtype
```

```
---  ------           --------------  -----
 0   branch_code      74 non-null     int64
 1   branch_name      74 non-null     object
 2   branch_location  74 non-null     object
dtypes: int64(1), object(2)
memory usage: 1.9+ KB
```

[14]:
```
# Inspeksi informasi umum pada data master_branch
# Mengubah tipe data branch_code dari int64 menjadi string
master_branch['branch_code'] = master_branch['branch_code'].astype(str)
# Cek kembali info DataFrame untuk memastikan sudah berubah
master_branch.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 74 entries, 0 to 73
Data columns (total 3 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   branch_code      74 non-null     object
 1   branch_name      74 non-null     object
 2   branch_location  74 non-null     object
dtypes: object(3)
memory usage: 1.9+ KB
```

Primary Key : branch_code Foreign Key : ... Dimensi Data : 74 baris dan 3 kolom

Pada data master_branch perlu dilakukan transformasi data pada kolom branch_code hal ini dilakukan karena perlu dirubah dari type int64 ke string

3 B. Transformasi & Integrasi Data

Task B.1 - Pastikan tidak ada redudansi data tiap primary key

[15]:
```
# Hapus duplikasi data transaction hingga master_branch pada tiap primary keynya
# Daftar tabel dan Primary Key masing-masing
tables_pk = {
    'transaction': ('transaction_id', transaction),
    'master_account': ('account_number', master_account),
    'master_customer': ('cif_number', master_customer),
    'master_transaction_code': ('transaction_code', master_transaction_code),
    'master_bank_code': ('bank_code', master_bank_code),
    'master_branch': ('branch_code', master_branch)
}

print("=== PENGECEKAN DUPLIKASI PRIMARY KEY ===\n")

for table_name, (pk_col, df) in tables_pk.items():
    # Mencari baris yang duplikat berdasarkan Primary Key (keep=False untuk␣
 ↪menampilkan semua baris duplikat)
```

```
    duplicates = df[df.duplicated(subset=[pk_col], keep=False)]

    print(f"--- Tabel: {table_name} (PK: '{pk_col}') ---")
    if len(duplicates) > 0:
        print(f" Ditemukan {len(duplicates)} baris duplikat!")
        display(duplicates)
    else:
        print(" Tidak ada duplikasi pada Primary Key.")
    print("-" * 50 + "\n")
```

```
=== PENGECEKAN DUPLIKASI PRIMARY KEY ===

--- Tabel: transaction (PK: 'transaction_id') ---
 Tidak ada duplikasi pada Primary Key.
--------------------------------------------------

--- Tabel: master_account (PK: 'account_number') ---
 Tidak ada duplikasi pada Primary Key.
--------------------------------------------------

--- Tabel: master_customer (PK: 'cif_number') ---
 Tidak ada duplikasi pada Primary Key.
--------------------------------------------------

--- Tabel: master_transaction_code (PK: 'transaction_code') ---
 Tidak ada duplikasi pada Primary Key.
--------------------------------------------------

--- Tabel: master_bank_code (PK: 'bank_code') ---
 Ditemukan 2 baris duplikat!
```

	bank_code	bank_swift_code	bank_name
19	116	PDJBIDJA	Bank Jatim
35	116	BPDKIDJA	Bank Aceh Syariah

```
--------------------------------------------------

--- Tabel: master_branch (PK: 'branch_code') ---
 Tidak ada duplikasi pada Primary Key.
--------------------------------------------------
```

[16]:
```
print("=== PROSES PEMBERSIHAN DUPLIKASI PRIMARY KEY ===\n")

# Menyimpan kembali DataFrame yang sudah dibersihkan ke variabel aslinya
for table_name, (pk_col, df) in tables_pk.items():
    initial_rows = len(df)
```

```
# Menghapus duplikasi, keep='first' berarti menyimpan baris pertama dan␣
 ↪menghapus duplikat setelahnya
    cleaned_df = df.drop_duplicates(subset=[pk_col], keep='first')

    final_rows = len(cleaned_df)
    removed_rows = initial_rows - final_rows
# Update kembali variabel DataFrame di workspace
    globals()[table_name] = cleaned_df

    print(f"--- Tabel: {table_name} ---")
    print(f"Jumlah baris awal  : {initial_rows}")
    print(f"Jumlah baris akhir : {final_rows}")
    if removed_rows > 0:
        print(f"  Berhasil menghapus {removed_rows} baris duplikat.")
    else:
        print("  Tidak ada duplikasi, data sudah bersih.")
    print("-" * 50 + "\n")
```

```
=== PROSES PEMBERSIHAN DUPLIKASI PRIMARY KEY ===

--- Tabel: transaction ---
Jumlah baris awal  : 1640
Jumlah baris akhir : 1640
  Tidak ada duplikasi, data sudah bersih.
--------------------------------------------------

--- Tabel: master_account ---
Jumlah baris awal  : 123
Jumlah baris akhir : 123
  Tidak ada duplikasi, data sudah bersih.
--------------------------------------------------

--- Tabel: master_customer ---
Jumlah baris awal  : 123
Jumlah baris akhir : 123
  Tidak ada duplikasi, data sudah bersih.
--------------------------------------------------

--- Tabel: master_transaction_code ---
Jumlah baris awal  : 6
Jumlah baris akhir : 6
  Tidak ada duplikasi, data sudah bersih.
--------------------------------------------------

--- Tabel: master_bank_code ---
Jumlah baris awal  : 38
Jumlah baris akhir : 37
```

```
 Berhasil menghapus 1 baris duplikat.
--------------------------------------------------

--- Tabel: master_branch ---
Jumlah baris awal  : 74
Jumlah baris akhir : 74
 Tidak ada duplikasi, data sudah bersih.
--------------------------------------------------
```

Task B.2 - Transformasi data transaction : transaction_date

- Konversikan kolom transaction_date yang semula berformat julian date menjadi date yang berformat YYYY-MM-DD sesuai ISO 8601 (https://www.iso.org/iso-8601-date-and-time-format.html)
- Contoh konversi : - Untuk tanggal 1 Januari 2025 maka format julian date dituliskan 2025001- Untuk tanggal 12 Januari 2025 maka format julian date dituliskan 2025012- Untuk tanggal 1 Februari 2025 maka format julian date dituliskan 2025032 (ditulis 032 karena 1 Februari adalah hari ke 32 dalam setahun)- dst.

[17]:
```python
# Transformasi kolom 'transaction_date' (format YYYYDDD ke datetime)
# Pastikan kolom transaction_date diubah menjadi tipe string terlebih dahulu
transaction['transaction_date'] = transaction['transaction_date'].astype(str)

# Konversi format Julian date (YYYYDDD) ke datetime standar menggunakan format␣
 ↪'%Y%j'
transaction['transaction_date'] = pd.to_datetime(
    transaction['transaction_date'].str[:4] + transaction['transaction_date'].
 ↪str[4:].str.zfill(3),
    format='%Y%j'
)

# Tampilkan beberapa baris pertama untuk verifikasi hasil konversi
print("=== Hasil Task B.2 (Konversi Transaction Date) ===")
display(transaction[['transaction_id', 'transaction_date']].head(5))

# Tampilkan informasi DataFrame terbaru
print("\nInformasi DataFrame terbaru:")
transaction.head(5)
```

```
=== Hasil Task B.2 (Konversi Transaction Date) ===
```

	transaction_id	transaction_date
0	TRX21325483579612	2025-08-01
1	TRX21335043336597	2025-08-01
2	TRX21343446066904	2025-08-01
3	TRX21357196529986	2025-08-01
4	TRX21439859637972	2025-08-02

Informasi DataFrame terbaru:

[17]:
```
      transaction_id  transaction_reference     account_number  \
0  TRX21325483579612  10000102025213254835 7  90000260801008087
1  TRX21335043336597  10000102025213350433 3  90000260801002198
2  TRX21343446066904  10000102025213434460 6  90000260801006643
3  TRX21357196529986  10000102025213571965 2  90000260801008644
4  TRX21439859637972  10000102025214398596 3  90000260802006117

  debit_or_credit transaction_code transaction_date  transaction_time  \
0               C           TRX001       2025-08-01            112726
1               C           TRX001       2025-08-01            142301
2               C           TRX001       2025-08-01            111501
3               C           TRX001       2025-08-01            105331
4               C           TRX001       2025-08-02            121626

    amount                           transaction_remark  channel
0  5200000    SETORTUNAI VIA CASH DEPOSIT MACHINE 00047  ATM_CDM
1  4000000                    SETORTUNAI VIA AGENT 00070    AGENT
2  7600000  SETORTUNAI VIA CASH RECYCLING MACHINE 00037  ATM_CRM
3  3100000                    SETORTUNAI VIA AGENT 00183    AGENT
4  8400000    SETORTUNAI VIA CASH DEPOSIT MACHINE 00070  ATM_CDM
```

Task B.2 - Transformasi data transaction : transaction_time

- Konversikan kolom transaction_time yang semula berformat integer menjadi time yang berformat HH:MM:SS
- Contoh konversi :- Transaksi dilakukan pukul 17:08:45 maka pada data dituliskan 170845- Transaksi dilakukan pukul 01:00:00 maka pada data dituliskan 10000- Transaksi dilakukan pukul 00:08:08 maka pada data dituliskan 808

[18]:
```
# Konversi tipe data transaction_time yang semula int menjadi string
transaction['transaction_time'] = transaction['transaction_time'].astype(str).
 ↪str.zfill(6)
transaction['transaction_time'] = (
    transaction['transaction_time'].str[:2] + ':' +
    transaction['transaction_time'].str[2:4] + ':' +
    transaction['transaction_time'].str[4:]
)
```

[19]:
```
# Tampilkan hasilnya
# Tampilkan informasi DataFrame terbaru
print("\nInformasi DataFrame terbaru:")
transaction.head()
```

Informasi DataFrame terbaru:

[19]:
```
      transaction_id  transaction_reference     account_number  \
0  TRX21325483579612  10000102025213254835 7  90000260801008087
1  TRX21335043336597  10000102025213350433 3  90000260801002198
2  TRX21343446066904  10000102025213434460 6  90000260801006643
3  TRX21357196529986  10000102025213571965 2  90000260801008644
4  TRX21439859637972  10000102025214398596 3  90000260802006117

  debit_or_credit transaction_code transaction_date transaction_time   amount  \
0               C           TRX001       2025-08-01         11:27:26  5200000
1               C           TRX001       2025-08-01         14:23:01  4000000
2               C           TRX001       2025-08-01         11:15:01  7600000
3               C           TRX001       2025-08-01         10:53:31  3100000
4               C           TRX001       2025-08-02         12:16:26  8400000

                           transaction_remark  channel
0    SETORTUNAI VIA CASH DEPOSIT MACHINE 00047  ATM_CDM
1                  SETORTUNAI VIA AGENT 00070    AGENT
2  SETORTUNAI VIA CASH RECYCLING MACHINE 00037  ATM_CRM
3                  SETORTUNAI VIA AGENT 00183    AGENT
4    SETORTUNAI VIA CASH DEPOSIT MACHINE 00070  ATM_CDM
```

Task B.3 - Transformasi data transaction : transaction_datetime

- Jika transaction_date sudah berformat YYYY-MM-DD dan transaction_time sudah berformat HH:MM:SS, lakukan assignment pada kolom transaction_date dengan menggabungkan kedua kolom tersebut sehingga formatnya menjadi YYYY-MM-DD HH:MM:SS
- Ubah nama kolom transaction_date menjadi transaction_datetime
- Konversi tipe data kolom transaction_datetime menjadi datetime
- Hapus kolom transaction_time

[20]:
```
# Gabungkan kolom transaction_date dan transaction_time
transaction['transaction_datetime'] = pd.to_datetime(
    transaction['transaction_date'].dt.strftime('%Y-%m-%d') + ' ' +␣
 ↪transaction['transaction_time']
)

# Konversi tipe datanya menjadi datetime
print("=== Cek Hasil Penggabungan ===")
display(transaction[['transaction_id', 'transaction_datetime']].head(5))

# Cek tipe data setelah transformasi
print("\nTipe data setelah transformasi:")
print(transaction[['transaction_datetime']].dtypes)

# Menghapus kolom 'transaction_date' dan 'transaction_time' yang lama
transaction = transaction.drop(columns=['transaction_date', 'transaction_time'])
```

```
# Tampilkan informasi DataFrame terbaru
print("\nInformasi DataFrame terbaru:")
transaction.info()
```

```
=== Cek Hasil Penggabungan ===

      transaction_id transaction_datetime
0  TRX21325483579612  2025-08-01 11:27:26
1  TRX21335043336597  2025-08-01 14:23:01
2  TRX21343446066904  2025-08-01 11:15:01
3  TRX21357196529986  2025-08-01 10:53:31
4  TRX21439859637972  2025-08-02 12:16:26


Tipe data setelah transformasi:
transaction_datetime    datetime64[ns]
dtype: object

Informasi DataFrame terbaru:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1640 entries, 0 to 1639
Data columns (total 9 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   transaction_id        1640 non-null   object
 1   transaction_reference 1640 non-null   object
 2   account_number        1640 non-null   int64
 3   debit_or_credit       1640 non-null   object
 4   transaction_code      1640 non-null   object
 5   amount                1640 non-null   int64
 6   transaction_remark    1640 non-null   object
 7   channel               1640 non-null   object
 8   transaction_datetime  1640 non-null   datetime64[ns]
dtypes: datetime64[ns](1), int64(2), object(6)
memory usage: 115.4+ KB
```

[21]:
```
transaction.head(5)
```

[21]:
```
      transaction_id  transaction_reference     account_number  \
0  TRX21325483579612  10000102025213254835 7  90000260801008087
1  TRX21335043336597  10000102025213350433 3  90000260801002198
2  TRX21343446066904  10000102025213434460 6  90000260801006643
3  TRX21357196529986  10000102025213571965 2  90000260801008644
4  TRX21439859637972  10000102025214398596 3  90000260802006117

  debit_or_credit transaction_code   amount  \
0               C           TRX001  5200000
1               C           TRX001  4000000
```

```
2               C           TRX001  7600000
3               C           TRX001  3100000
4               C           TRX001  8400000

                            transaction_remark  channel transaction_datetime
0    SETORTUNAI VIA CASH DEPOSIT MACHINE 00047  ATM_CDM  2025-08-01 11:27:26
1                   SETORTUNAI VIA AGENT 00070    AGENT  2025-08-01 14:23:01
2  SETORTUNAI VIA CASH RECYCLING MACHINE 00037  ATM_CRM  2025-08-01 11:15:01
3                   SETORTUNAI VIA AGENT 00183    AGENT  2025-08-01 10:53:31
4    SETORTUNAI VIA CASH DEPOSIT MACHINE 00070  ATM_CDM  2025-08-02 12:16:26
```

Task B.4 - Transformasi data transaction : Urutkan data transaction berdasarkan account_number dan transaction_datetime (keduanya secara ascending)

[22]:
```
# Urutkan data transaction sesuai ketentuan
# Mengurutkan DataFrame transaction berdasarkan account_number dan␣
 ↪transaction_datetime secara ascending
transaction = transaction.sort_values(
    by=['account_number', 'transaction_datetime'],
    ascending=[True, True]
).reset_index(drop=True)

# Tampilkan hasilnya
print("=== Hasil Pengurutan Transaction ===")
display(transaction[['account_number', 'transaction_datetime',␣
 ↪'transaction_id', 'amount']].head(5))
```

```
=== Hasil Pengurutan Transaction ===
```

```
      account_number transaction_datetime     transaction_id   amount
0  90000260801002198  2025-08-01 14:23:01  TRX21335043336597  4000000
1  90000260801002198  2025-08-03 06:12:57  TRX21568992585422   493544
2  90000260801002198  2025-08-03 06:12:57  TRX21568992587949     7500
3  90000260801002198  2025-08-04 23:51:16  TRX21637494499456    34001
4  90000260801002198  2025-08-08 11:52:51  TRX22056308589193  2005000
```

Task B.5 - Transformasi data transaction : Pisahkan amount credit dan debit

- Buat kolom baru yang diberi nama credit_amount lalu isikan dengan amount jika kolom debit_or_credit berisi kode kredit selainnya isikan 0
- Buat kolom baru yang diberi nama debit_amount lalu isikan dengan amount jika kolom debit_or_credit berisi kode debit selainnya isikan 0
- Jika dipastikan sudah terisi, hapus kolom amount

[23]:
```
import numpy as np
# Buat kolom credit_amount lalu isikan nilainya sesuai ketentuan
# Buat kolom credit_amount: isi dengan amount jika debit_or_credit berisi 'C'␣
 ↪(Kredit), selain itu isi 0
```

```
transaction['credit_amount'] = np.where(transaction['debit_or_credit'] == 'C',␣
 ↪transaction['amount'], 0)

# Buat kolom debit_amount lalu isikan nilainya sesuai ketentuan
# Buat kolom debit_amount: isi dengan amount jika debit_or_credit berisi 'D'␣
 ↪(Debit), selain itu isi 0
transaction['debit_amount'] = np.where(transaction['debit_or_credit'] == 'D',␣
 ↪transaction['amount'], 0)

# Hapus kolom amount yang lama
transaction = transaction.drop(columns=['amount'])

# Tampilkan beberapa baris hasil untuk memastikan sudah sesuai
print("=== Hasil Task B.5 (Pemisahan Credit & Debit Amount) ===")
display(transaction[['transaction_id', 'debit_or_credit', 'credit_amount',␣
 ↪'debit_amount', 'transaction_datetime']].head(5))

# Tampilkan informasi DataFrame terbaru
print("\nInformasi DataFrame terbaru:")
transaction.info()
```

```
=== Hasil Task B.5 (Pemisahan Credit & Debit Amount) ===
```

	transaction_id	debit_or_credit	credit_amount	debit_amount	transaction_datetime
0	TRX21335043336597	C	4000000	0	2025-08-01 14:23:01
1	TRX21568992585422	D	0	493544	2025-08-03 06:12:57
2	TRX21568992587949	D	0	7500	2025-08-03 06:12:57
3	TRX21637494499456	D	0	34001	2025-08-04 23:51:16
4	TRX22056308589193	C	2005000	0	2025-08-08 11:52:51

```
Informasi DataFrame terbaru:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1640 entries, 0 to 1639
Data columns (total 10 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   transaction_id         1640 non-null   object
 1   transaction_reference  1640 non-null   object
 2   account_number         1640 non-null   int64
 3   debit_or_credit        1640 non-null   object
```

```
 4   transaction_code        1640 non-null   object
 5   transaction_remark      1640 non-null   object
 6   channel                 1640 non-null   object
 7   transaction_datetime    1640 non-null   datetime64[ns]
 8   credit_amount           1640 non-null   int64
 9   debit_amount            1640 non-null   int64
dtypes: datetime64[ns](1), int64(3), object(6)
memory usage: 128.3+ KB
```

Task B.6 - Integrasi data transaction : Hubungkan data transaction dengan master_transaction_code

[24]:
```
# Hubungkan data transaksi dengan master kode transaksi
# Hubungkan data transaction dengan master_transaction_code menggunakan kolom 'transaction_code'
transaction = pd.merge(
    transaction,
    master_transaction_code,
    on='transaction_code',
    how='left'
)

#Tampilkan beberapa baris hasil penggabungan
print("=== Hasil Integrasi Task B.6 ===")
display(transaction[['transaction_id', 'transaction_code', 'transaction_desc', 'credit_amount', 'debit_amount']].head(5))
```

```
=== Hasil Integrasi Task B.6 ===
```

	transaction_id	transaction_code	transaction_desc	credit_amount	debit_amount
0	TRX21335043336597	TRX001	Cash Deposit	4000000	0
1	TRX21568992585422	TRX002	Transfer	0	493544
2	TRX21568992587949	TRX092	Administration Fee	0	7500
3	TRX21637494499456	TRX002	Transfer	0	34001
4	TRX22056308589193	TRX002	Transfer	2005000	0

[25]:
```
#Hitung dan tampilkan banyak transaksi per masing-masing jenis (transaction_desc)
print("\n=== Jumlah Transaksi Berdasarkan Jenisnya ===")
transaction_counts = transaction['transaction_desc'].value_counts()
print(transaction_counts)
```

```
=== Jumlah Transaksi Berdasarkan Jenisnya ===
transaction_desc
Transfer              1000
Administration Fee     301
Cash Deposit           141
Monthly Fee            137
Cash Withdrawal         61
Name: count, dtype: int64
```

Task B.7 - Transformasi data transaction : Biaya Tambahan

- Perhatikan transaksi dengan kode **TRX092** (kode untuk Administration Fee). Buat variabel baru yang diberi nama `transaction_fee` lalu ambil kolom transaction_reference dan debit_amount saja dari data transaction
- Ubah penamaan kolom debit_amount menjadi additional_fee pada variabel `transaction_fee`
- Pada data `transaction` hapus baris data yang kodenya **TRX092**
- Lakukan penggabungan data `transaction` dengan `transaction_fee` dengan kunci transaction_reference
- Jika transaksi tidak dikenakan biaya maka isikan 0 pada kolom additional_fee

[26]:
```
# Buat variabel transaction_fee khusus untuk kode 'TRX092' dan ambil kolom yang␣
 ↪dibutuhkan
transaction_fee = transaction.loc[
    transaction['transaction_code'] == 'TRX092',
    ['transaction_reference', 'debit_amount']
].copy()

# Ubah nama kolom debit_amount menjadi additional_fee secara eksplisit
transaction_fee = transaction_fee.rename(columns={'debit_amount':␣
 ↪'additional_fee'})

# Gabungkan (merge) kembali ke DataFrame utama transaction berdasarkan␣
 ↪'transaction_reference'
transaction = pd.merge(transaction, transaction_fee,␣
 ↪on='transaction_reference', how='left')

# Isikan 0 pada kolom additional_fee untuk baris yang tidak memiliki biaya␣
 ↪tambahan (NaN)
transaction['additional_fee'] = transaction['additional_fee'].fillna(0)

# Tampilkan hasil untuk verifikasi
print("=== Hasil Task B.7 (Biaya Tambahan) ===")
display(transaction[['transaction_id', 'transaction_code', 'additional_fee']].
 ↪head(10))
```

```
=== Hasil Task B.7 (Biaya Tambahan) ===
```

```
      transaction_id transaction_code  additional_fee
0  TRX21335043336597           TRX001             0.0
1  TRX21568992585422           TRX002          7500.0
2  TRX21568992587949           TRX092          7500.0
3  TRX21637494499456           TRX002             0.0
4  TRX22056308589193           TRX002             0.0
5  TRX22117759052275           TRX002             0.0
6  TRX22212472817476           TRX002             0.0
7  TRX22279857174397           TRX002             0.0
8  TRX22270627153785           TRX002             0.0
9  TRX22239965806598           TRX002             0.0
```

Task B.8 - Transformasi data transaction : Saldo Berjalan!

- Buat kolom baru pada data `transaction` yang diberi nama balance
- Kolom balance adalah 'Saldo Berjalan' yang terurut berdasarkan tanggal dan waktu transaksi. Jika transaksi merupakan transaksi kredit artinya uang masuk ke rekening maka saldo akan bertambah sebaliknya transaksi debit artinya uang keluar dari rekening maka saldo akan berkurang
- Perhatikan juga bahwa biaya tambahan menjadi pengurang saldo

[27]:
```
# Hitung perubahan nilai transaksi bersih per baris terlebih dahulu
net_transaction = transaction['credit_amount'] - transaction['debit_amount'] -␣
 ↪transaction['additional_fee']

# Buat kolom 'balance' sebagai akumulasi (cumsum) per masing-masing nomor␣
 ↪rekening berdasarkan net_transaction
transaction['balance'] = net_transaction.groupby(transaction['account_number']).
 ↪cumsum()

# Proses sort data agar terurut berdasarkan account_number dan␣
 ↪transaction_datetime secara ascending
transaction = transaction.sort_values(by=['account_number',␣
 ↪'transaction_datetime']).reset_index(drop=True)

# Tampilkan beberapa baris hasil untuk verifikasi
print("=== Hasil Task B.8 (Saldo Berjalan / Balance) ===")
display(transaction[['account_number', 'transaction_datetime', 'credit_amount',␣
 ↪'debit_amount', 'additional_fee', 'balance']].head(10))

# Tampilkan informasi DataFrame terbaru
print("\nInformasi DataFrame terbaru:")
transaction.info()
```

```
=== Hasil Task B.8 (Saldo Berjalan / Balance) ===
```

```
      account_number transaction_datetime  credit_amount  debit_amount  \
0  90000260801002198  2025-08-01 14:23:01        4000000             0
```

```
1  90000260801002198  2025-08-03 06:12:57              0        493544
2  90000260801002198  2025-08-03 06:12:57              0          7500
3  90000260801002198  2025-08-04 23:51:16              0         34001
4  90000260801002198  2025-08-08 11:52:51        2005000             0
5  90000260801002198  2025-08-09 12:44:59              0         52656
6  90000260801002198  2025-08-10 06:34:32              0        598415
7  90000260801002198  2025-08-10 07:34:20              0         67562
8  90000260801002198  2025-08-10 18:59:33              0         96525
9  90000260801002198  2025-08-10 20:28:59              0         44354

   additional_fee    balance
0             0.0  4000000.0
1          7500.0  3498956.0
2          7500.0  3483956.0
3             0.0  3449955.0
4             0.0  5454955.0
5             0.0  5402299.0
6             0.0  4803884.0
7             0.0  4736322.0
8             0.0  4639797.0
9             0.0  4595443.0


Informasi DataFrame terbaru:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1640 entries, 0 to 1639
Data columns (total 13 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   transaction_id        1640 non-null   object
 1   transaction_reference 1640 non-null   object
 2   account_number        1640 non-null   int64
 3   debit_or_credit       1640 non-null   object
 4   transaction_code      1640 non-null   object
 5   transaction_remark    1640 non-null   object
 6   channel               1640 non-null   object
 7   transaction_datetime  1640 non-null   datetime64[ns]
 8   credit_amount         1640 non-null   int64
 9   debit_amount          1640 non-null   int64
 10  transaction_desc      1640 non-null   object
 11  additional_fee        1640 non-null   float64
 12  balance               1640 non-null   float64
dtypes: datetime64[ns](1), float64(2), int64(3), object(7)
memory usage: 166.7+ KB
```

Task B.9 - Transformasi data transaction : Transmode Code (Bagian 1)

- Buat kolom baru pada data `transaction` yang diberi nama transmode_code

- Kolom transmode_code merupakan penanda detail sebuah transaksi
- Isikan kolom transmode_code dengan : - SETOR : Jika transaksi merupakan transaksi setor tunai via channel apapun - TARIK : Jika transaksi merupakan transaksi tarik tunai via channel apapun - PURCHASE : Jika transaksi merupakan transaksi pembelian barang - INTER : Jika transaksi merupakan transaksi internal (biaya bulanan, pajak, bagi hasil, bonus, dll)

[28]:
```
# Proses kalkulasi transmode_code
# Identifikasi transaction_reference yang muncul lebih dari 1 kali (khas untuk␣
 ↪transaksi Overbooking / TRX002)
ref_counts = transaction['transaction_reference'].value_counts()
overbooking_refs = ref_counts[ref_counts >= 2].index

# Definisikan kondisi pengelompokan
conditions = [
    # Setor, Tarik, Purchase, Inter berdasarkan deskripsi transaksi
    transaction['transaction_desc'].str.contains('SETOR|DEPOSIT', case=False,␣
 ↪na=False),
    transaction['transaction_desc'].str.contains('TARIK|WITHDRAWAL',␣
 ↪case=False, na=False),
    transaction['transaction_desc'].str.contains('PURCHASE|PEMBELIAN',␣
 ↪case=False, na=False),
    transaction['transaction_desc'].str.
 ↪contains('BIAYA|PAJAK|INTEREST|BONUS|ADMIN|BUNGA', case=False, na=False),

    # Overbooking (TRX002 dengan reference ganda / antar rekening internal FSB␣
 ↪Bank)
    (transaction['transaction_code'] == 'TRX002') &␣
 ↪(transaction['transaction_reference'].isin(overbooking_refs)),

    # Incoming (Transfer masuk / dana masuk dari luar)
    (transaction['transaction_code'] == 'TRX002') &␣
 ↪(transaction['debit_or_credit'] == 'C'),

    # Outgoing (Transfer keluar / dana keluar ke luar)
    (transaction['transaction_code'] == 'TRX002') &␣
 ↪(transaction['debit_or_credit'] == 'D')
]

choices = ['SETOR', 'TARIK', 'PURCHASE', 'INTER', 'OVERBOOKING', 'INCOMING',␣
 ↪'OUTGOING']

#Buat kolom baru transmode_code menggunakan np.select
transaction['transmode_code'] = np.select(conditions, choices, default='OTHER')

#Tampilkan hasil verifikasi
print("=== Hasil Task B.9 (Transmode Code) ===")
```

```
display(transaction[['transaction_id', 'transaction_code', 'transaction_desc',␣
 ↪'debit_or_credit', 'transmode_code']].head(10))

#Tampilkan distribusi jumlah masing-masing transmode_code
print("\nDistribusi Jumlah Transmode Code:")
print(transaction['transmode_code'].value_counts())

# Tampilkan informasi DataFrame terbaru
print("\nInformasi DataFrame terbaru:")
transaction.info()
```

```
=== Hasil Task B.9 (Transmode Code) ===

      transaction_id transaction_code    transaction_desc debit_or_credit  \
0  TRX21335043336597           TRX001        Cash Deposit               C
1  TRX21568992585422           TRX002            Transfer               D
2  TRX21568992587949           TRX092  Administration Fee               D
3  TRX21637494499456           TRX002            Transfer               D
4  TRX22056308589193           TRX002            Transfer               C
5  TRX22117759052275           TRX002            Transfer               D
6  TRX22212472817476           TRX002            Transfer               D
7  TRX22279857174397           TRX002            Transfer               D
8  TRX22270627153785           TRX002            Transfer               D
9  TRX22239965806598           TRX002            Transfer               D

  transmode_code
0          SETOR
1    OVERBOOKING
2          INTER
3       OUTGOING
4       INCOMING
5       OUTGOING
6       OUTGOING
7       OUTGOING
8       OUTGOING
9       OUTGOING


Distribusi Jumlah Transmode Code:
transmode_code
OUTGOING       664
INTER          301
OVERBOOKING    266
SETOR          141
OTHER          137
INCOMING        70
TARIK           61
Name: count, dtype: int64
```

```
Informasi DataFrame terbaru:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1640 entries, 0 to 1639
Data columns (total 14 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   transaction_id        1640 non-null   object
 1   transaction_reference 1640 non-null   object
 2   account_number        1640 non-null   int64
 3   debit_or_credit       1640 non-null   object
 4   transaction_code      1640 non-null   object
 5   transaction_remark    1640 non-null   object
 6   channel               1640 non-null   object
 7   transaction_datetime  1640 non-null   datetime64[ns]
 8   credit_amount         1640 non-null   int64
 9   debit_amount          1640 non-null   int64
 10  transaction_desc      1640 non-null   object
 11  additional_fee        1640 non-null   float64
 12  balance               1640 non-null   float64
 13  transmode_code        1640 non-null   object
dtypes: datetime64[ns](1), float64(2), int64(3), object(8)
memory usage: 179.5+ KB
```

Task B.9 - Transformasi data transaction : Transmode Code (Bagian 2)

- Transaksi OVERBOOKING adalah transaksi yang dilakukan dari dan ke rekening bank yang sama yakni FSB Bank (FSB Bank memiliki kode : 456). Ciri khas transaksi ini adalah 2 buah baris memiliki kode transaction_reference yang sama dan kode transaksinya adalah TRX002. Memiliki dua buah baris dikarenakan satu rekening untuk mentransfer dana (kode : Debit) dan satunya untuk menerima dana (kode : Credit)
- Transaksi INCOMING adalah transaksi keuangan berupa aliran dana atau uang yang masuk dan diterima ke dalam suatu rekening, sehingga menambah jumlah saldo.
- Transaksi OUTGOING adalah transaksi keuangan berupa aliran dana atau uang yang keluar dan dikirim dari suatu rekening, sehingga mengurangi jumlah saldo.
- Isikan kolom transmode_code dengan : - OVERBOOKING : Jika transaksi merupakan transaksi overbooking dengan ketentuan diatas - INCOMING : Jika transaksi merupakan transaksi masuk dari bank lain ke rekening FSB Bank - OUTGOING : Jika transaksi merupakan transaksi keluar dari rekening FSB Bank ke bank lain

[29]:
```
# Periksa kolom yang memiliki kode transaksi TRX002
# Periksa kolom yang memiliki kode transaksi TRX002 dan hitung frekuensi␣
 ↪kemunculan transaction_reference
transfer_transaction = transaction[transaction['transaction_code'] == 'TRX002']

# Lakukan perhitungan jumlah kemunculan per transaction_reference
transfer_transaction = transfer_transaction.groupby(['transaction_code',␣
 ↪'transaction_reference']).size().reset_index(name='total_transaction')
```

```
# Tampilkan hasilnya untuk verifikasi
print("=== Hasil Perhitungan TRX002 ===")
display(transfer_transaction.head(10))
```

```
=== Hasil Perhitungan TRX002 ===
```

	transaction_code	transaction_reference	total_transaction
0	TRX002	200002020252152593076	1
1	TRX002	200002020252154894887	1
2	TRX002	200002020252156899258	1
3	TRX002	200002020252163749449	1
4	TRX002	200002020252164553385	1
5	TRX002	200002020252165345304	1
6	TRX002	200002020252167461101	1
7	TRX002	200002020252173934811	1
8	TRX002	200002020252174719385	1
9	TRX002	200002020252174982763	1

[30]:
```
# Ambil transaction_reference yang muncul lebih dari 1 kali sebagai list
# Ambil transaction_reference yang muncul lebih dari 1 kali sebagai list
ref_counts = transaction['transaction_reference'].value_counts()
overbooking_refs = ref_counts[ref_counts > 1].index.tolist()

# Proses kalkulasi transmode_code
conditions = [
    transaction['transaction_code'] == 'TRX002',
    transaction['transaction_code'] == 'TRX001',
    transaction['transaction_code'] == 'TRX003'
]

# Definisikan aturan penentuan berdasarkan kondisi di atas atau gabungkan␣
 ↪dengan pengecekan reference ganda
transaction['transmode_code'] = np.select(
    [
        (transaction['transaction_code'] == 'TRX002') &␣
 ↪(transaction['transaction_reference'].isin(overbooking_refs)),
        (transaction['transaction_code'] == 'TRX002') &␣
 ↪(transaction['debit_or_credit'] == 'C'),
        (transaction['transaction_code'] == 'TRX002') &␣
 ↪(transaction['debit_or_credit'] == 'D'),
        transaction['transaction_desc'].str.contains('SETOR|DEPOSIT',␣
 ↪case=False, na=False),
        transaction['transaction_desc'].str.contains('TARIK|WITHDRAWAL',␣
 ↪case=False, na=False),
        transaction['transaction_desc'].str.contains('PURCHASE|PEMBELIAN',␣
 ↪case=False, na=False)
```

```
    ],
    ['OVERBOOKING', 'INCOMING', 'OUTGOING', 'SETOR', 'TARIK', 'INTER'],
    default='OTHER'
)

# Hitung banyak transaksinya tiap transmode_code
transmode_summary = transaction['transmode_code'].value_counts().reset_index()
transmode_summary.columns = ['transmode_code', 'count']
display(transmode_summary)
# Proses kalkulasi transmode_code
transaction['trasmode_code'] = ...
```

	transmode_code	count
0	OUTGOING	664
1	OTHER	438
2	OVERBOOKING	266
3	SETOR	141
4	INCOMING	70
5	TARIK	61

Task B.10 - Transformasi data transaction : Acquirer Bank Code!

- Acquirer Bank Code : Kode bank yang bertindak sebagai *acquirer*, yaitu bank atau institusi keuangan yang menerima dan memproses transaksi dari sisi merchant
- Pada data transaction, buat kolom baru `acquirer_bank_code` lalu isikan dengan kode bank dilakukannya transaksi
- Ketentuan : - Untuk transaksi **TARIK** (Cash Withdrawal) : `TARIK VIA ATM <acquirer_bank_code> <lokasi>`. Contoh : TARIK VIA ATM 121 KUPANG maka transaksi penarikan tunai dilakukan di atm bank dengan kode 121 di Kupang, NTT (ambil kode 121) - Untuk transaksi **OUTGOING** tipe **QRIS** : `QRIS_<acquirer_bank_code>_<retrieval_code>_<account_name>` Contoh : QRIS_028_48084_Bengkel Express, maka telah dilakukan pembayaran dengan QRIS dengan tujuan kode bank 028 dengan nama merchant (account name) Bengkel Express - Untuk transaksi **TRANSFER** (INCOMING, OUTGOING atau OVERBOOKING) tipe **Non QRIS** : `TRF_<acquirer_bank_code>_<account_number>_<account_name>` atau `BIFAST_<acquirer_bank_code>_<account_number>_<account_name>` Contoh : TRF_200_200735184920673_Mila Kartik dengan kode Credit berarti transfer masuk dari bank dengan kode 200 dan nomor rekening 200735184920673 atas nama Mila Kartik. BIFAST_124_2787558867_Imam Bonjol dengan kode Debit berarti transfer keluar dari bank dengan kode 124 dan nomor rekening 2787558867 atas nama Imam Bonjol

[31]:
```
from datetime import datetime

# 1. Samakan tipe data kolom penghubung (account_number) menjadi string di␣
 ↪kedua DataFrame
transaction['account_number'] = transaction['account_number'].astype(str)
master_account['account_number'] = master_account['account_number'].astype(str)
```

```
# (Opsional jika cif_number juga berbeda tipenya)
if 'cif_number' in transaction.columns and 'cif_number' in master_customer.
 ↪columns:
    transaction['cif_number'] = transaction['cif_number'].astype(str)
    master_customer['cif_number'] = master_customer['cif_number'].astype(str)

# 2. Lakukan kembali proses Merge dengan master_account
df_merged = transaction.merge(
    master_account[['account_number', 'cif_number', 'account_open_date']],
    on='account_number',
    how='left'
)

# 3. Lanjutkan merge dengan master_customer menggunakan cif_number
df_merged = df_merged.merge(
    master_customer,
    on='cif_number',
    how='left'
)

# 4. Perhitungan Account Aging
df_merged['account_open_date'] = pd.to_datetime(df_merged['account_open_date'],␣
 ↪errors='coerce')
df_merged['transaction_datetime'] = pd.
 ↪to_datetime(df_merged['transaction_datetime'], errors='coerce')
df_merged['account_aging_days'] = (df_merged['transaction_datetime'] -␣
 ↪df_merged['account_open_date']).dt.days

# 5. Penambahan Kolom Metadata Audit
df_merged['created_date'] = datetime.now()
df_merged['created_by'] = 'ETL_PIPELINE_ADMIN'

print("Berhasil melakukan merge!")
display(df_merged[['transaction_id', 'account_number', 'account_aging_days',␣
 ↪'created_date']].head())
```

```
Berhasil melakukan merge!
```

	transaction_id	account_number	account_aging_days	\
0	TRX21335043336597	90000260801002198	0	
1	TRX21568992585422	90000260801002198	2	
2	TRX21568992587949	90000260801002198	2	
3	TRX21637494499456	90000260801002198	3	
4	TRX22056308589193	90000260801002198	7	

	created_date
0	2026-10-20 08:21:42.166507
1	2026-10-20 08:21:42.166507

```
2 2026-10-20 08:21:42.166507
3 2026-10-20 08:21:42.166507
4 2026-10-20 08:21:42.166507
```

[32]:
```
print(df.columns)
```

```
Index(['branch_code', 'branch_name', 'branch_location'], dtype='object')
```

Task B.11 - Transformasi data transaction : Acquirer Account Number!

- Acquirer Bank Code : Nomor rekening yang terkait dengan pihak acquirer dalam suatu transaksi atau bisa disebut juga nomor rekening lawan (*counterparty account*)
- Pada data transaction, buat kolom baru `acquirer_account_number` lalu isikan dengan nomor rekening lawan transaksi
- Ketentuan : - Untuk transaksi **TRANSFER** (INCOMING, OUTGOING atau OVERBOOKING) tipe **Non QRIS** : `TRF_<acquirer_bank_code>_<account_number>_<account_name>` atau `BIFAST_<acquirer_bank_code>_<account_number>_<account_name>` Contoh :TRF_200_200735184920673_Mila Kartik dengan kode Credit berarti transfer masuk dari bank dengan kode 200 dan nomor rekening 200735184920673 atas nama Mila Kartik (ambil nomor rekeningnya). BIFAST_124_2787558867_Imam Bonjol dengan kode Debit berarti transfer keluar dari bank dengan kode 124 dan nomor rekening 2787558867 atas nama Imam Bonjol (ambil nomor rekeningnya).

[33]:
```
import re
# Lakukan perhitungan dan pengkondisian
# Fungsi untuk mengekstrak nomor rekening lawan (acquirer account number) dari teks remark/deskripsi
def extract_acquirer_account(row):
    remark = str(row.get('transaction_remark', ''))
    match = re.search(r'(?:TRF|BIFAST)_\d+_(\d+)_', remark)
    if match:
        return match.group(1)
    return None

# Terapkan fungsi ekstraksi ke DataFrame utama
source_col = 'transaction_remark' if 'transaction_remark' in df_merged.columns else 'transaction_desc'
df_merged['acquirer_account_number'] = df_merged.apply(extract_acquirer_account, axis=1)

# Tampilkan hasilnya
print(f"Jumlah baris terdeteksi acquirer account: {df_merged['acquirer_account_number'].notnull().sum()}")
display(df_merged[['transaction_id', source_col, 'acquirer_account_number']].head(10))
```

```
Jumlah baris terdeteksi acquirer account: 558
```

```
      transaction_id                          transaction_remark  \
0  TRX21335043336597                   SETORTUNAI VIA AGENT 00070
1  TRX21568992585422              TRF_032_6587603669_Nadia Safitri
2  TRX21568992587949              TRF_032_6587603669_Nadia Safitri
3  TRX21637494499456  QRIS_451_90190_Angkringan Express Sudirman
4  TRX22056308589193         TRF_200_200735184920673_Mila Kartik
5  TRX22117759052275                  QRIS_506_75915_Toko Kencana
6  TRX22212472817476                QRIS_112_77710_Anugerah Optik
7  TRX22279857174397      QRIS_118_83662_Apotek Podomoro Menteng
8  TRX22270627153785                  QRIS_547_94643_Corner Klinik
9  TRX22239965806598                QRIS_506_56706_Shop Abadi Kuta

  acquirer_account_number
0                    None
1              6587603669
2              6587603669
3                    None
4         200735184920673
5                    None
6                    None
7                    None
8                    None
9                    None
```

Task B.12 - Transformasi data transaction : Acquirer Account Name!

- Acquirer Bank Code : Nama pemilik rekening yang terkait dengan pihak acquirer dalam suatu transaksi atau bisa disebut juga nama pemilik rekening lawan (*counterparty account name*)
- Pada data transaction, buat kolom baru `acquirer_account_name` lalu isikan dengan nama pemilik rekening lawan transaksi
- Ketentuan : - Untuk transaksi **TRANSFER** (INCOMING, OUTGOING atau OVERBOOKING) tipe **Non QRIS** : `TRF_<acquirer_bank_code>_<account_number>_<account_name>` atau `BIFAST_<acquirer_bank_code>_<account_number>_<account_name>` Contoh :TRF_200_200735184920673_Mila Kartik dengan kode Credit berarti transfer masuk dari bank dengan kode 200 dan nomor rekening 200735184920673 atas nama Mila Kartik (ambil nama pemilik rekeningnya). BIFAST_124_2787558867_Imam Bonjol dengan kode Debit berarti transfer keluar dari bank dengan kode 124 dan nomor rekening 2787558867 atas nama Imam Bonjol (ambil nama pemilik rekeningnya).

[34]:
```
# Lakukan perhitungan dan pengkondisian
import re
# Fungsi untuk mengekstrak acquirer account name dari transaction_remark
def extract_acquirer_account_name(row):
    remark = str(row.get('transaction_remark', ''))
    # Pola regex untuk menangkap format:␣
 ↪TRF_<bank_code>_<account_number>_<account_name>
```

```
    match = re.search(r'(?:TRF|BIFAST)_\d+_\d+_(.+)', remark)
    if match:
        return match.group(1)
    return None

# Terapkan fungsi ke DataFrame
df_merged['acquirer_account_name'] = df_merged.
 ↪apply(extract_acquirer_account_name, axis=1)

# Verifikasi dan tampilkan hasil
print(f"Jumlah baris terdeteksi acquirer account name:␣
 ↪{df_merged['acquirer_account_name'].notnull().sum()}")
display(df_merged[['transaction_id', 'transaction_remark',␣
 ↪'acquirer_account_name']].head(5))
```

```
Jumlah baris terdeteksi acquirer account name: 558
```

```
      transaction_id                          transaction_remark  \
0  TRX21335043336597                  SETORTUNAI VIA AGENT 00070
1  TRX21568992585422            TRF_032_6587603669_Nadia Safitri
2  TRX21568992587949            TRF_032_6587603669_Nadia Safitri
3  TRX21637494499456  QRIS_451_90190_Angkringan Express Sudirman
4  TRX22056308589193         TRF_200_200735184920673_Mila Kartik

  acquirer_account_name
0                  None
1         Nadia Safitri
2         Nadia Safitri
3                  None
4           Mila Kartik
```

[35]:
```
# Tampilkan hasilnya
print(f"Jumlah baris terdeteksi acquirer account name:␣
 ↪{df_merged['acquirer_account_name'].notnull().sum()}")
df_merged.head(5)
```

```
Jumlah baris terdeteksi acquirer account name: 558
```

[35]:
```
      transaction_id  transaction_reference     account_number  \
0  TRX21335043336597  100001020252133504333  90000260801002198
1  TRX21568992585422  200002020252156899258  90000260801002198
2  TRX21568992587949  200002020252156899258  90000260801002198
3  TRX21637494499456  200002020252163749449  90000260801002198
4  TRX22056308589193  200002020252205630858  90000260801002198

  debit_or_credit transaction_code  \
0               C           TRX001
1               D           TRX002
```

```
2               D           TRX092
3               D           TRX002
4               C           TRX002

                          transaction_remark    channel transaction_datetime  \
0                  SETORTUNAI VIA AGENT 00070      AGENT  2025-08-01 14:23:01
1            TRF_032_6587603669_Nadia Safitri        ATM  2025-08-03 06:12:57
2            TRF_032_6587603669_Nadia Safitri     SYSTEM  2025-08-03 06:12:57
3  QRIS_451_90190_Angkringan Express Sudirman  DGBANKING  2025-08-04 23:51:16
4         TRF_200_200735184920673_Mila Kartik  DGBANKING  2025-08-08 11:52:51

   credit_amount  debit_amount  … customer_city  customer_phone  \
0        4000000             0  …       Bandung      0872801911
1              0        493544  …       Bandung      0872801911
2              0          7500  …       Bandung      0872801911
3              0         34001  …       Bandung      0872801911
4        2005000             0  …       Bandung      0872801911

         occupation married_status           customer_email  \
0  Ibu Rumah Tangga         Single  kania_puspita@gmail.com
1  Ibu Rumah Tangga         Single  kania_puspita@gmail.com
2  Ibu Rumah Tangga         Single  kania_puspita@gmail.com
3  Ibu Rumah Tangga         Single  kania_puspita@gmail.com
4  Ibu Rumah Tangga         Single  kania_puspita@gmail.com

  account_aging_days               created_date          created_by  \
0                  0 2026-10-20 08:21:42.166507  ETL_PIPELINE_ADMIN
1                  2 2026-10-20 08:21:42.166507  ETL_PIPELINE_ADMIN
2                  2 2026-10-20 08:21:42.166507  ETL_PIPELINE_ADMIN
3                  3 2026-10-20 08:21:42.166507  ETL_PIPELINE_ADMIN
4                  7 2026-10-20 08:21:42.166507  ETL_PIPELINE_ADMIN

  acquirer_account_number acquirer_account_name
0                    None                  None
1              6587603669         Nadia Safitri
2              6587603669         Nadia Safitri
3                    None                  None
4         200735184920673           Mila Kartik

[5 rows x 34 columns]
```

Task B.13 - Integrasi data transaction : Hubungkan data transaction dengan master_bank_code

- Integrasikan data `transaction` dengan `master_bank_code` untuk mendapatkan kolom bank_name atau `acquirer_bank_name`

[36]:
```
# Proses integrasi master_bank_code
def extract_acquirer_bank_code(row):
    source_col = 'transaction_remark' if 'transaction_remark' in row.index else␣
 ↪'transaction_desc'
    remark = str(row.get(source_col, ''))
    match = re.search(r'(?:TRF|BIFAST)_(\d+)_', remark)
    if match:
        return match.group(1)
    return None

if 'acquirer_bank_code' not in df_merged.columns:
    df_merged['acquirer_bank_code'] = df_merged.
 ↪apply(extract_acquirer_bank_code, axis=1)

# Samakan tipe data kolom penghubung (tipe string)
df_merged['acquirer_bank_code'] = df_merged['acquirer_bank_code'].astype(str)
master_bank_code['bank_code'] = master_bank_code['bank_code'].astype(str)

# Lakukan merge (left join) dengan master_bank_code
df_merged = df_merged.merge(
    master_bank_code[['bank_code', 'bank_name', 'bank_swift_code']],
    left_on='acquirer_bank_code',
    right_on='bank_code',
    how='left',
    suffixes=('', '_master')
)

# Ubah nama kolom 'bank_name' menjadi 'acquirer_bank_name'
df_merged.rename(columns={'bank_name': 'acquirer_bank_name'}, inplace=True)

# Bersihkan kolom duplikat yang tidak perlu
if 'bank_code_master' in df_merged.columns:
    df_merged.drop(columns=['bank_code_master'], inplace=True)
if 'bank_code' in df_merged.columns and 'acquirer_bank_code' in df_merged.
 ↪columns:
    df_merged.drop(columns=['bank_code'], errors='ignore', inplace=True)

# Tampilkan hasil
print(f"Total baris setelah integrasi: {len(df_merged)}")
display(df_merged[['transaction_id', 'acquirer_bank_code',␣
 ↪'acquirer_bank_name']].head(10))
```

```
Total baris setelah integrasi: 1640
```

```
/tmp/ipykernel_16918/3080046919.py:15: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

```
See the caveats in the documentation: https://pandas.pydata.org/pandas-
docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  master_bank_code['bank_code'] = master_bank_code['bank_code'].astype(str)
```

	transaction_id	acquirer_bank_code	acquirer_bank_name
0	TRX21335043336597	None	NaN
1	TRX21568992585422	032	Bank Jago
2	TRX21568992587949	032	Bank Jago
3	TRX21637494499456	None	NaN
4	TRX22056308589193	200	NaN
5	TRX22117759052275	None	NaN
6	TRX22212472817476	None	NaN
7	TRX22279857174397	None	NaN
8	TRX22270627153785	None	NaN
9	TRX22239965806598	None	NaN

[37]:
```
print(f"Total baris setelah integrasi: {len(df_merged)}")
df_merged.head(5)
```

```
Total baris setelah integrasi: 1640
```

[37]:

	transaction_id	transaction_reference	account_number	debit_or_credit	transaction_code	transaction_remark	channel	transaction_datetime	credit_amount	debit_amount	…	married_status	customer_email
0	TRX21335043336597	100001020252133504333	90000260801002198	C	TRX001	SETORTUNAI VIA AGENT 00070	AGENT	2025-08-01 14:23:01	4000000	0	…	Single	kania_puspita@gmail.com
1	TRX21568992585422	200002020252156899258	90000260801002198	D	TRX002	TRF_032_6587603669_Nadia Safitri	ATM	2025-08-03 06:12:57	0	493544	…	Single	kania_puspita@gmail.com
2	TRX21568992587949	200002020252156899258	90000260801002198	D	TRX092	TRF_032_6587603669_Nadia Safitri	SYSTEM	2025-08-03 06:12:57	0	7500	…	Single	kania_puspita@gmail.com
3	TRX21637494499456	200002020252163749449	90000260801002198	D	TRX002	QRIS_451_90190_Angkringan Express Sudirman	DGBANKING	2025-08-04 23:51:16	0	34001	…	Single	kania_puspita@gmail.com
4	TRX22056308589193	200002020252205630858	90000260801002198	C	TRX002	TRF_200_200735184920673_Mila Kartik	DGBANKING	2025-08-08 11:52:51	2005000	0	…	Single	kania_puspita@gmail.com

```
   account_aging_days                created_date          created_by  \
0                   0 2026-10-20 08:21:42.166507  ETL_PIPELINE_ADMIN
1                   2 2026-10-20 08:21:42.166507  ETL_PIPELINE_ADMIN
2                   2 2026-10-20 08:21:42.166507  ETL_PIPELINE_ADMIN
3                   3 2026-10-20 08:21:42.166507  ETL_PIPELINE_ADMIN
4                   7 2026-10-20 08:21:42.166507  ETL_PIPELINE_ADMIN

  acquirer_account_number acquirer_account_name acquirer_bank_code  \
0                    None                  None                None
1              6587603669         Nadia Safitri                 032
2              6587603669         Nadia Safitri                 032
3                    None                  None                None
4         200735184920673           Mila Kartik                 200

  acquirer_bank_name bank_swift_code
0                NaN              NaN
1          Bank Jago         BABEIDJA
2          Bank Jago         BABEIDJA
3                NaN              NaN
4                NaN              NaN

[5 rows x 37 columns]
```

Task B.14 - Integrasi data transaction : Hubungkan data transaction dengan master_account

- Integrasikan data `transaction` dengan `master_account` untuk mendapatkan kolom cif_number, account_open_date, account_currency_type, account_status, account_product hingga product_name
- Buat kolom baru bernama account_aging yang merupakan selisih antara tanggal transaksi dengan tanggal buka rekening dalam hari

[38]:
```
# Proses integrasi
df_merged['account_number'] = df_merged['account_number'].astype(str)
master_account['account_number'] = master_account['account_number'].astype(str)

# Proses integrasi (merge) left join dengan master_account
# Mengambil kolom yang diperlukan dari master_account
cols_to_merge = ['account_number', 'cif_number', 'account_open_date',␣
 ↪'account_currency_type', 'account_status', 'product_code', 'product_name']

df_merged = df_merged.merge(
    master_account[cols_to_merge],
    on='account_number',
    how='left',
    suffixes=('', '_master')
```

```
)

# Proses kalkulasi account_aging (selisih hari antara tanggal transaksi dengan␣
 ↪tanggal buka rekening)
df_merged['account_open_date'] = pd.to_datetime(df_merged['account_open_date'],␣
 ↪errors='coerce')
df_merged['transaction_datetime'] = pd.
 ↪to_datetime(df_merged['transaction_datetime'], errors='coerce')

df_merged['account_aging'] = (df_merged['transaction_datetime'] -␣
 ↪df_merged['account_open_date']).dt.days

# Tampilkan hasilnya
print(f"Total baris setelah integrasi master_account: {len(df_merged)}")
display(df_merged[['transaction_id', 'account_number', 'account_open_date',␣
 ↪'transaction_datetime', 'account_aging', 'product_name', 'account_status']].
 ↪head(10))
```

```
Total baris setelah integrasi master_account: 1640
```

	transaction_id	account_number	account_open_date	transaction_datetime	account_aging	product_name	account_status
0	TRX21335043336597	90000260801002198	2025-08-01	2025-08-01 14:23:01	0	TABUNGAN UTAMA GOLD	ACTIVE
1	TRX21568992585422	90000260801002198	2025-08-01	2025-08-03 06:12:57	2	TABUNGAN UTAMA GOLD	ACTIVE
2	TRX21568992587949	90000260801002198	2025-08-01	2025-08-03 06:12:57	2	TABUNGAN UTAMA GOLD	ACTIVE
3	TRX21637494499456	90000260801002198	2025-08-01	2025-08-04 23:51:16	3	TABUNGAN UTAMA GOLD	ACTIVE
4	TRX22056308589193	90000260801002198	2025-08-01	2025-08-08 11:52:51	7	TABUNGAN UTAMA GOLD	ACTIVE
5	TRX22117759052275	90000260801002198	2025-08-01	2025-08-09 12:44:59	8	TABUNGAN UTAMA GOLD	ACTIVE
6	TRX22212472817476	90000260801002198	2025-08-01	2025-08-10 06:34:32	9	TABUNGAN UTAMA GOLD	ACTIVE
7	TRX22279857174397	90000260801002198	2025-08-01	2025-08-10 07:34:20	9	TABUNGAN UTAMA GOLD	ACTIVE
8	TRX22270627153785	90000260801002198	2025-08-01	2025-08-10 18:59:33	9	TABUNGAN UTAMA GOLD	ACTIVE
9	TRX22239965806598	90000260801002198	2025-08-01	2025-08-10 20:28:59	9	TABUNGAN UTAMA GOLD	ACTIVE

[39]:
```
df_merged.head(5)
```

[39]:

	transaction_id	transaction_reference	account_number	debit_or_credit	transaction_code	transaction_remark	channel	transaction_datetime	credit_amount	debit_amount	…	acquirer_bank_code	acquirer_bank_name	bank_swift_code	cif_number_master	account_open_date_master	account_currency_type	account_status	product_code	product_name	account_aging
0	TRX21335043336597	100001020252133504333	90000260801002198	C	TRX001	SETORTUNAI VIA AGENT 00070	AGENT	2025-08-01 14:23:01	4000000	0	…	None	NaN	NaN	CIF092706	2025-08-01	IDR	ACTIVE	TABGLDMRB	TABUNGAN UTAMA GOLD	0
1	TRX21568992585422	200002020252156899258	90000260801002198	D	TRX002	TRF_032_6587603669_Nadia Safitri	ATM	2025-08-03 06:12:57	0	493544	…	032	Bank Jago	BABEIDJA	CIF092706	2025-08-01	IDR	ACTIVE	TABGLDMRB	TABUNGAN UTAMA GOLD	2
2	TRX21568992587949	200002020252156899258	90000260801002198	D	TRX092	TRF_032_6587603669_Nadia Safitri	SYSTEM	2025-08-03 06:12:57	0	7500	…	032	Bank Jago	BABEIDJA	CIF092706	2025-08-01	IDR	ACTIVE	TABGLDMRB	TABUNGAN UTAMA GOLD	2
3	TRX21637494499456	200002020252163749449	90000260801002198	D	TRX002	QRIS_451_90190_Angkringan Express Sudirman	DGBANKING	2025-08-04 23:51:16	0	34001	…	None	NaN	NaN	CIF092706	2025-08-01	IDR	ACTIVE	TABGLDMRB	TABUNGAN UTAMA GOLD	3
4	TRX22056308589193	200002020252205630858	90000260801002198	C	TRX002	TRF_200_200735184920673_Mila Kartik	DGBANKING	2025-08-08 11:52:51	2005000	0	…	200	NaN	NaN	CIF092706	2025-08-01	IDR	ACTIVE	TABGLDMRB	TABUNGAN UTAMA GOLD	0

```
4             7

[5 rows x 44 columns]
```

Task B.15 - Integrasi data transaction : Hubungkan data transaction dengan master_customer dan master_branch

- Integrasikan data `transaction` dengan `master_customer` untuk mendapatkan kolom cif_number, customer_name, gender dan branch_code
- Setelahnya, integrasikan data `transaction` dengan `master_branch` untuk mendapatkan kolom branch_code dan branch_name

[40]:
```python
# Proses integrasi
if 'cif_number' in df_merged.columns and 'cif_number' in master_customer.
 ↪columns:
    df_merged['cif_number'] = df_merged['cif_number'].astype(str)
    master_customer['cif_number'] = master_customer['cif_number'].astype(str)

# Integrasi data dengan master_customer
cols_customer = ['cif_number', 'customer_name', 'gender', 'branch_code']
df_merged = df_merged.merge(
    master_customer[[c for c in cols_customer if c in master_customer.columns]],
    on='cif_number',
    how='left',
    suffixes=('', '_customer')
)

# Menyesuaikan kolom branch_code jika terjadi duplikasi dari tabel sebelumnya
if 'branch_code_customer' in df_merged.columns:
    df_merged['branch_code'] = df_merged['branch_code'].
 ↪fillna(df_merged['branch_code_customer'])
    df_merged.drop(columns=['branch_code_customer'], inplace=True)

# Pastikan tipe data branch_code konsisten sebelum merge dengan master_branch
df_merged['branch_code'] = df_merged['branch_code'].astype(str)
master_branch['branch_code'] = master_branch['branch_code'].astype(str)

# Integrasi data dengan master_branch untuk mendapatkan branch_name
cols_branch = ['branch_code', 'branch_name']
df_merged = df_merged.merge(
    master_branch[[c for c in cols_branch if c in master_branch.columns]],
    on='branch_code',
    how='left',
    suffixes=('', '_branch')
)

# Bersihkan kolom duplikat tambahan jika ada
```

```
if 'branch_code_branch' in df_merged.columns:
    df_merged.drop(columns=['branch_code_branch'], inplace=True)

# Tampilkan hasilnya
print(f"Total baris setelah integrasi master_customer dan master_branch:␣
 ↪{len(df_merged)}")
display(df_merged[['transaction_id', 'cif_number', 'customer_name', 'gender',␣
 ↪'branch_code', 'branch_name']].head(10))
df_merged.head(5)
```

```
Total baris setelah integrasi master_customer dan master_branch: 1640
```

	transaction_id	cif_number	customer_name	gender	branch_code	branch_name
0	TRX21335043336597	CIF092706	Kania Puspita	P	20001	KC Bandung
1	TRX21568992585422	CIF092706	Kania Puspita	P	20001	KC Bandung
2	TRX21568992587949	CIF092706	Kania Puspita	P	20001	KC Bandung
3	TRX21637494499456	CIF092706	Kania Puspita	P	20001	KC Bandung
4	TRX22056308589193	CIF092706	Kania Puspita	P	20001	KC Bandung
5	TRX22117759052275	CIF092706	Kania Puspita	P	20001	KC Bandung
6	TRX22212472817476	CIF092706	Kania Puspita	P	20001	KC Bandung
7	TRX22279857174397	CIF092706	Kania Puspita	P	20001	KC Bandung
8	TRX22270627153785	CIF092706	Kania Puspita	P	20001	KC Bandung
9	TRX22239965806598	CIF092706	Kania Puspita	P	20001	KC Bandung

[40]:

	transaction_id	transaction_reference	account_number	debit_or_credit	transaction_code	transaction_remark	channel	transaction_datetime	credit_amount	debit_amount	…	cif_number_master
0	TRX21335043336597	100001020252133504333	90000260801002198	C	TRX001	SETORTUNAI VIA AGENT 00070	AGENT	2025-08-01 14:23:01	4000000	0	…	CIF092706
1	TRX21568992585422	200002020252156899258	90000260801002198	D	TRX002	TRF_032_6587603669_Nadia Safitri	ATM	2025-08-03 06:12:57				
2	TRX21568992587949	200002020252156899258	90000260801002198	D	TRX092	TRF_032_6587603669_Nadia Safitri	SYSTEM	2025-08-03 06:12:57				
3	TRX21637494499456	200002020252163749449	90000260801002198	D	TRX002	QRIS_451_90190_Angkringan Express Sudirman	DGBANKING	2025-08-04 23:51:16				
4	TRX22056308589193	200002020252205630858	90000260801002198	C	TRX002	TRF_200_200735184920673_Mila Kartik	DGBANKING	2025-08-08 11:52:51				

```
1              0        493544  …         CIF092706
2              0          7500  …         CIF092706
3              0         34001  …         CIF092706
4        2005000             0  …         CIF092706

   account_open_date_master  account_currency_type account_status  \
0                2025-08-01                    IDR         ACTIVE
1                2025-08-01                    IDR         ACTIVE
2                2025-08-01                    IDR         ACTIVE
3                2025-08-01                    IDR         ACTIVE
4                2025-08-01                    IDR         ACTIVE

  product_code         product_name account_aging customer_name_customer  \
0    TABGLDMRB  TABUNGAN UTAMA GOLD             0          Kania Puspita
1    TABGLDMRB  TABUNGAN UTAMA GOLD             2          Kania Puspita
2    TABGLDMRB  TABUNGAN UTAMA GOLD             2          Kania Puspita
3    TABGLDMRB  TABUNGAN UTAMA GOLD             3          Kania Puspita
4    TABGLDMRB  TABUNGAN UTAMA GOLD             7          Kania Puspita

  gender_customer branch_name
0               P  KC Bandung
1               P  KC Bandung
2               P  KC Bandung
3               P  KC Bandung
4               P  KC Bandung

[5 rows x 47 columns]
```

Task B.16 - Transformasi data transaction : Tambahkan Metadata Tanggal dan Created By!

- Buat kolom created_date isikan dengan tanggal dan jam saat ini
- Buat kolom created_by isikan dengan 'SYSTEM' (or you can put your name on it :D)

[41]:
```python
# Tambahkan metadata
# Menambahkan kolom metadata tanggal dan user pembuat
df_merged['created_date'] = datetime.now()
df_merged['created_by'] = 'NUROCHMAN'

# Verifikasi hasil penambahan metadata
print(f"Total kolom setelah Task B.16: {len(df_merged.columns)}")
display(df_merged[['transaction_id', 'created_date', 'created_by']].head(5))
```

```
Total kolom setelah Task B.16: 47

      transaction_id               created_date created_by
0  TRX21335043336597 2026-10-20 08:21:42.627084  NUROCHMAN
1  TRX21568992585422 2026-10-20 08:21:42.627084  NUROCHMAN
```

```
2  TRX21568992587949 2026-10-20 08:21:42.627084  NUROCHMAN
3  TRX21637494499456 2026-10-20 08:21:42.627084  NUROCHMAN
4  TRX22056308589193 2026-10-20 08:21:42.627084  NUROCHMAN
```

4 C. Data Profiling & Validation

- Lakukan instalasi library data-profiling dengan mengeksekusi perintah `pip install fg-data-profiling`
- Lalu lakukan data profiling, simpan ke dalam html dan tampilkan hasilnya
- Periksa apakah semuanya sudah sesuai?

[42]:
```
!pip install fg-data-profiling
```

```
Requirement already satisfied: fg-data-profiling in
/opt/conda/lib/python3.11/site-packages (4.20.0)
Requirement already satisfied: scipy<1.19.0,>=1.8 in
/opt/conda/lib/python3.11/site-packages (from fg-data-profiling) (1.11.3)
Requirement already satisfied: pandas!=1.4.0,<3.0,>1.5 in
/opt/conda/lib/python3.11/site-packages (from fg-data-profiling) (2.0.3)
Requirement already satisfied: matplotlib<3.12,>=3.5 in
/opt/conda/lib/python3.11/site-packages (from fg-data-profiling) (3.8.0)
Requirement already satisfied: pydantic<3,>=2 in /opt/conda/lib/python3.11/site-
packages (from fg-data-profiling) (2.14.1)
Requirement already satisfied: PyYAML<6.1,>=6.0.3 in
/opt/conda/lib/python3.11/site-packages (from fg-data-profiling) (6.0.3)
Requirement already satisfied: jinja2<3.2,>=3.1.6 in
/opt/conda/lib/python3.11/site-packages (from fg-data-profiling) (3.1.6)
Requirement already satisfied: visions<0.9,>=0.7.5 in
/opt/conda/lib/python3.11/site-packages (from
visions[type_geometry,type_image_path]<0.9,>=0.7.5->fg-data-profiling) (0.8.2)
Requirement already satisfied: numpy<2.6,>=1.22 in
/opt/conda/lib/python3.11/site-packages (from fg-data-profiling) (1.26.4)
Requirement already satisfied: minify-html>=0.15.0 in
/opt/conda/lib/python3.11/site-packages (from fg-data-profiling) (0.18.1)
Requirement already satisfied: filetype>=1.0.0 in
/opt/conda/lib/python3.11/site-packages (from fg-data-profiling) (1.2.0)
Requirement already satisfied: phik<0.13,>=0.12.5 in
/opt/conda/lib/python3.11/site-packages (from fg-data-profiling) (0.12.5)
Requirement already satisfied: requests<3,>=2.32.0 in
/opt/conda/lib/python3.11/site-packages (from fg-data-profiling) (2.34.2)
Requirement already satisfied: tqdm<5,>=4.66.3 in
/opt/conda/lib/python3.11/site-packages (from fg-data-profiling) (4.70.1)
Requirement already satisfied: seaborn<0.14,>=0.10.1 in
/opt/conda/lib/python3.11/site-packages (from fg-data-profiling) (0.13.0)
Requirement already satisfied: multimethod<2,>=1.4 in
/opt/conda/lib/python3.11/site-packages (from fg-data-profiling) (1.12)
Requirement already satisfied: statsmodels<1,>=0.13.2 in
/opt/conda/lib/python3.11/site-packages (from fg-data-profiling) (0.14.0)
```

```
Requirement already satisfied: typeguard<5,>=4 in
/opt/conda/lib/python3.11/site-packages (from fg-data-profiling) (4.6.0)
Requirement already satisfied: imagehash==4.3.2 in
/opt/conda/lib/python3.11/site-packages (from fg-data-profiling) (4.3.2)
Requirement already satisfied: wordcloud>=1.9.4 in
/opt/conda/lib/python3.11/site-packages (from fg-data-profiling) (1.9.6)
Requirement already satisfied: dacite<2,>=1.9 in /opt/conda/lib/python3.11/site-
packages (from fg-data-profiling) (1.9.2)
Requirement already satisfied: numba<0.68,>=0.60 in
/opt/conda/lib/python3.11/site-packages (from fg-data-profiling) (0.67.0)
Requirement already satisfied: PyWavelets in /opt/conda/lib/python3.11/site-
packages (from imagehash==4.3.2->fg-data-profiling) (1.4.1)
Requirement already satisfied: pillow in /opt/conda/lib/python3.11/site-packages
(from imagehash==4.3.2->fg-data-profiling) (10.1.0)
Requirement already satisfied: MarkupSafe>=2.0 in
/opt/conda/lib/python3.11/site-packages (from jinja2<3.2,>=3.1.6->fg-data-
profiling) (2.1.3)
Requirement already satisfied: contourpy>=1.0.1 in
/opt/conda/lib/python3.11/site-packages (from matplotlib<3.12,>=3.5->fg-data-
profiling) (1.1.1)
Requirement already satisfied: cycler>=0.10 in /opt/conda/lib/python3.11/site-
packages (from matplotlib<3.12,>=3.5->fg-data-profiling) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in
/opt/conda/lib/python3.11/site-packages (from matplotlib<3.12,>=3.5->fg-data-
profiling) (4.43.1)
Requirement already satisfied: kiwisolver>=1.0.1 in
/opt/conda/lib/python3.11/site-packages (from matplotlib<3.12,>=3.5->fg-data-
profiling) (1.4.5)
Requirement already satisfied: packaging>=20.0 in
/opt/conda/lib/python3.11/site-packages (from matplotlib<3.12,>=3.5->fg-data-
profiling) (23.2)
Requirement already satisfied: pyparsing>=2.3.1 in
/opt/conda/lib/python3.11/site-packages (from matplotlib<3.12,>=3.5->fg-data-
profiling) (3.1.1)
Requirement already satisfied: python-dateutil>=2.7 in
/opt/conda/lib/python3.11/site-packages (from matplotlib<3.12,>=3.5->fg-data-
profiling) (2.8.2)
Requirement already satisfied: llvmlite<0.50,>=0.49.0dev0 in
/opt/conda/lib/python3.11/site-packages (from numba<0.68,>=0.60->fg-data-
profiling) (0.49.0)
Requirement already satisfied: pytz>=2020.1 in /opt/conda/lib/python3.11/site-
packages (from pandas!=1.4.0,<3.0,>1.5->fg-data-profiling) (2023.3.post1)
Requirement already satisfied: tzdata>=2022.1 in /opt/conda/lib/python3.11/site-
packages (from pandas!=1.4.0,<3.0,>1.5->fg-data-profiling) (2023.3)
Requirement already satisfied: joblib>=0.14.1 in /opt/conda/lib/python3.11/site-
packages (from phik<0.13,>=0.12.5->fg-data-profiling) (1.3.2)
Requirement already satisfied: annotated-types>=0.6.0 in
/opt/conda/lib/python3.11/site-packages (from pydantic<3,>=2->fg-data-profiling)
```

```
(0.8.0)
Requirement already satisfied: pydantic-core==2.50.1 in
/opt/conda/lib/python3.11/site-packages (from pydantic<3,>=2->fg-data-profiling)
(2.50.1)
Requirement already satisfied: typing-extensions>=4.16.0 in
/opt/conda/lib/python3.11/site-packages (from pydantic<3,>=2->fg-data-profiling)
(4.16.0)
Requirement already satisfied: typing-inspection>=0.4.4 in
/opt/conda/lib/python3.11/site-packages (from pydantic<3,>=2->fg-data-profiling)
(0.4.4)
Requirement already satisfied: charset_normalizer<4,>=2 in
/opt/conda/lib/python3.11/site-packages (from requests<3,>=2.32.0->fg-data-
profiling) (3.3.0)
Requirement already satisfied: idna<4,>=2.5 in /opt/conda/lib/python3.11/site-
packages (from requests<3,>=2.32.0->fg-data-profiling) (3.4)
Requirement already satisfied: urllib3<3,>=1.26 in
/opt/conda/lib/python3.11/site-packages (from requests<3,>=2.32.0->fg-data-
profiling) (2.0.7)
Requirement already satisfied: certifi>=2023.5.7 in
/opt/conda/lib/python3.11/site-packages (from requests<3,>=2.32.0->fg-data-
profiling) (2023.7.22)
Requirement already satisfied: patsy>=0.5.2 in /opt/conda/lib/python3.11/site-
packages (from statsmodels<1,>=0.13.2->fg-data-profiling) (0.5.3)
Requirement already satisfied: attrs>=19.3.0 in /opt/conda/lib/python3.11/site-
packages (from
visions<0.9,>=0.7.5->visions[type_geometry,type_image_path]<0.9,>=0.7.5->fg-
data-profiling) (23.1.0)
Requirement already satisfied: networkx>=2.4 in /opt/conda/lib/python3.11/site-
packages (from
visions<0.9,>=0.7.5->visions[type_geometry,type_image_path]<0.9,>=0.7.5->fg-
data-profiling) (3.2)
Requirement already satisfied: puremagic in /opt/conda/lib/python3.11/site-
packages (from
visions<0.9,>=0.7.5->visions[type_geometry,type_image_path]<0.9,>=0.7.5->fg-
data-profiling) (1.30)
Requirement already satisfied: shapely in /opt/conda/lib/python3.11/site-
packages (from visions[type_geometry,type_image_path]<0.9,>=0.7.5->fg-data-
profiling) (2.2.0)
Requirement already satisfied: six in /opt/conda/lib/python3.11/site-packages
(from patsy>=0.5.2->statsmodels<1,>=0.13.2->fg-data-profiling) (1.16.0)
```

[43]:
```
# Hitung dan tampilkan banyak transaksi per masing - masing jenisnya
df_summary = df_merged.groupby(
    ['transaction_code', 'transaction_desc', 'trasmode_code']
).size().reset_index(name='count')

# Urutkan dari jumlah transaksi terbanyak
```

```
df_summary = df_summary.sort_values(by='count', ascending=False)

# Tampilkan hasilnya
display(df_summary)
```

	transaction_code	transaction_desc	trasmode_code	count
1	TRX002	Transfer	Ellipsis	1000
4	TRX092	Administration Fee	Ellipsis	301
0	TRX001	Cash Deposit	Ellipsis	141
3	TRX091	Monthly Fee	Ellipsis	137
2	TRX003	Cash Withdrawal	Ellipsis	61

[44]:
```
!pip install "numpy<2.0"
```

```
Requirement already satisfied: numpy<2.0 in /opt/conda/lib/python3.11/site-
packages (1.26.4)
```

[45]:
```
from data_profiling import ProfileReport

# Gunakan minimal=True agar proses jauh lebih cepat dan tidak hang
profile = ProfileReport(df_merged, title="FSB Bank E-Statement Data Mart␣
 ↪Profiling Report", minimal=True)

# Simpan report sebagai interactive HTML file
profile.to_file("fsb_bank_datamart_profile.html")

print("Laporan data profiling berhasil dibuat dan disimpan!")
```

```
Summarize dataset:   0%|          | 0/5 [00:00<?, ?it/s]


  0%|          | 0/47 [00:00<?, ?it/s]
 49%|        | 23/47 [00:00<00:00, 136.46it/s]
100%|      | 47/47 [00:00<00:00, 115.59it/s]

Generate report structure:   0%|          | 0/1 [00:00<?, ?it/s]

Render HTML:   0%|          | 0/1 [00:00<?, ?it/s]

Export report to file:   0%|          | 0/1 [00:00<?, ?it/s]

Laporan data profiling berhasil dibuat dan disimpan!
```

5 D. Load Data to Database

- Setelah semua proses berhasil dilakukan, simpan data `transaction` ke database dengan nama tabel `f_estatement_transaction`
- Periksa apakah datanya sudah berhasil di-load

[47]:
```
!pip install psycopg2-binary
```

```
Collecting psycopg2-binary
  Downloading psycopg2_binary-2.9.13-cp311-cp311-
manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (4.9 kB)
Downloading psycopg2_binary-2.9.13-cp311-cp311-
manylinux2014_x86_64.manylinux_2_17_x86_64.whl (4.3 MB)
                         4.3/4.3 MB
1.7 MB/s eta 0:00:0000:0100:01
Installing collected packages: psycopg2-binary
Successfully installed psycopg2-binary-2.9.13
```

[53]:
```python
from sqlalchemy import create_engine
import pandas as pd

# Hapus kolom trasmode_code jika masih ada
if 'trasmode_code' in df_merged.columns:
    df_merged = df_merged.drop(columns=['trasmode_code'])

# Ubah DB_NAME menjadi 'postgres' agar sesuai dengan CloudBeaver
DB_USER = 'airflow'
DB_PASSWORD = 'airflow'
DB_HOST = 'postgres'
DB_PORT = '5432'
DB_NAME = 'postgres'  # <-- Ubah ke 'postgres'

connection_string = f"postgresql://{DB_USER}:{DB_PASSWORD}@{DB_HOST}:{DB_PORT}/
 ↪{DB_NAME}"
engine = create_engine(connection_string)

# Load DataFrame ke database PostgreSQL
df_merged.to_sql('f_estatement_transaction', con=engine, if_exists='replace',␣
 ↪index=False)

print("Data berhasil di-load ke database postgres!")
```

```
Data berhasil di-load ke database postgres!
```
